ICT103 Programming

Topic 6
Functions and Modules

CIHE Australia

Resources

Prescribed Texts

- McMullen, K., Matthews, E., & Parsons, J. J. (2022). Programming with Python (1st ed.)., Cengage Learning

Recommended Readings

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media, Inc

Lesson Learned from Week 5:

- Create and use Lists, Dictionaries, and DataFrames
- Perform traversal on lists, dictionaries, and dataframes
- Apply common functions/operations
- Understand real-life analogies for each structure

Week 6: Lesson Learning Outcomes

- Differentiate between built-in functions, imported functions, and programmer-defined functions.
- State the purpose of a function's return value.
- Explain the meaning of "pass by value" and "pass by reference" in the context of functions

- Describe module structures
- Explain how the keyword import copies code from another module.
- Explain how to create modules
- Understand the purpose of the modules in Python

Introduction

- Functions help organize code into reusable blocks.
- Modules help organize code into separate files for reuse.

- Benefits:
  - Reusability
  - Easier debugging
  - Better code readability

Code without functions

```
print("Hello,
  Python!")
```

→

Code with functions

```
def main():
  print("Hello,
    Python!")
main()
```

→

Code organized in modules

```
greetings.py
def greet():
  print("Hello,
    Python!")
main()
```

```
main.py
import greetings
greetings.greet()
```

Function Basics

- A function is a named block of program code that performs a specific task
- Three categories:
  - Built-in functions:
    - Provided by a programming language
  - Imported functions:
    - Packaged in modules and added to a program using the keyword `import`
  - Programmer-defined functions:
    - Created by programmer

Function Signature

```
def function_name(parameters):
    """Optional docstring"""
    # code here
```

- A part of the function that uniquely defines a function for the compiler or interpreter

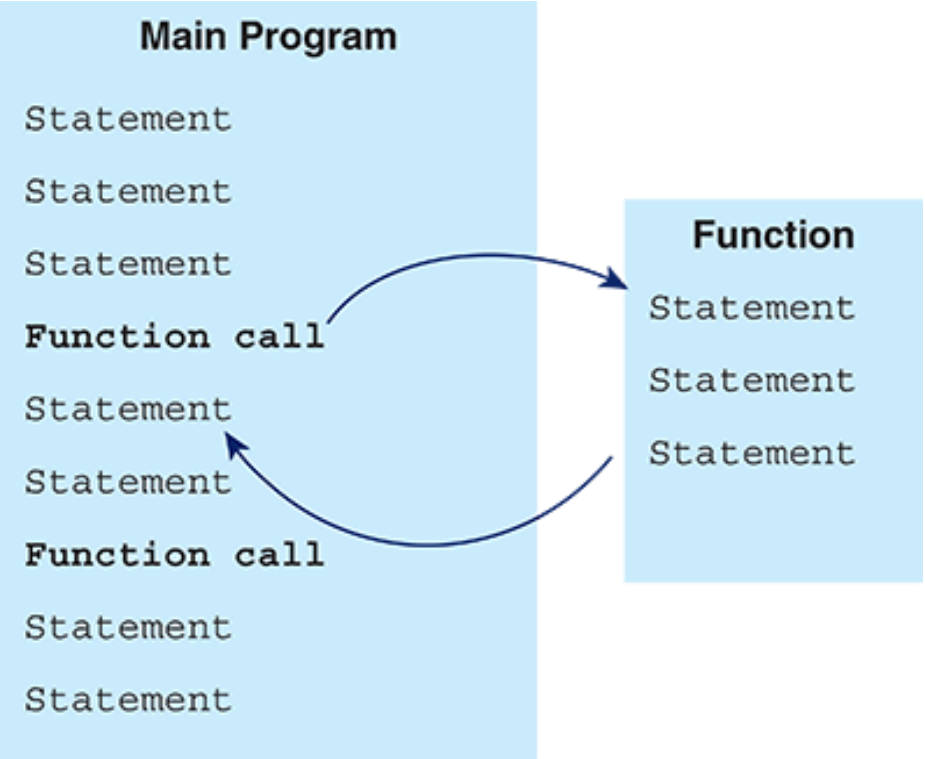
```
# Function definition
def display():
    """This is the body of the function display()"""
    print("Hello Python!!!")
```

- Components are:
  - The keyword `def`
  - The function name
  - Optional: The function parameters (the parameter list)

- As like variable names, you can't have two functions with the same names
  - Even if the parameter lists are different

Flow of Execution

- A function is triggered by a function call, potentially from multiple locations in a program.
- Function calls transfer execution to the specified function; perform the statements it contains, and then flow returns to the previous execution path




```python
# Function definition
def display():
    """This is the body of the function display()"""
    print("Hello Python!!!")

# Function Call
display()
print("End of logic..")
```

Output

```
Hello Python!!!
End of logic..
```

Functions Parameters vs Arguments

- Parameters:
  - Variables defined in the function declaration; act as placeholders for input values.
- Arguments:
  - Actual values passed to the function when it is called.

```python
# Function definition with parameters
def disp_message(name, age):
    print(f"Hello {name}, you are {age} years old.")

# Function call with arguments
disp_message("Jessie", 25)
```

Parameters

Arguments

Function part	Example	Type
Parameters	name, age	Placeholder
Arguments	“Jessie”, 25	Actual value

Output

```
Hello Jessie, you are 25 years old.
```

Return Values

- Functions can send values back to the caller using `return`
- In Python, void functions return the value `None`, representing nothing returned

```python
def add(a, b):
    return a + b

result = add(5, 3)
print("Addition is: ",result)  # Output: 8
```

Output

```
Addition is:  8
```

Multiple-Return Values

- A function can return more than one value by separating them with commas.
- The returned values are actually packed as tuples
  - A tuple is an ordered, immutable sequence.

```python
def get_person_info():
    name = "Sally"
    age = 25
    city = "Sydney"
    return name, age, city

# Call the function
person_name, person_age, person_city = get_person_info()

print(person_name)  # Output: Alice
print(person_age)   # Output: 25
print(person_city)  # Output: Sydney
```

Tuple

Output

```
Sally
25
Sydney
```

Functions with Default Arguments

- You can provide default values for function parameters to use it if the function is called without an argument for that parameter

```python
def welcome_user(name="Guest"):
    print(f"Hello, {name}!")

# Function call without argument
welcome_user()          # Output: Hello, Guest!

# Function call with an argument
welcome_user("Bob")     # Output: Hello, Bob!
```

Default value

Output

```
Hello, Guest!
Hello, Bob!
```

Scope

- A scope refers to the visibility of program components to other parts of the program

- Local variable:
  - A variable whose scope is limited only inside the function

- Global variable:
  - A variable accessible to an entire program
  - Defined outside of any function in the source code
  - Usually defined at the beginning of the program

```python
x = 10  # global

def example():
    y = 5  # local
    print(x, y)

example()  # Output: 10 5
# print(y)  # Error: y is not defined
```

Function Arguments – Mutable and Immutable

- In Python, all arguments are passed by object reference.
- Immutable objects (int, float, string, tuple)
  - Cannot be changed inside function
- Mutable objects (list, dict, set)
  - Can be modified inside function

Example: Immutable

```python
def increment(x):
    x = x + 1
    print("Inside function:", x)

num = 5
increment(num)
print("Outside function:", num)
```

Output

```
Inside function: 6
Outside function: 5
```

Example: Mutable

```python
def add_item(lst):
    lst.append(4)
    print("Inside function:", lst)

my_list = [1, 2, 3]
add_item(my_list)
print("Outside function:", my_list)
```

Output

```
Inside function: [1, 2, 3, 4]
Outside function: [1, 2, 3, 4]
```

Modules

- Modules are pre-programmed pieces of code written by other programmers that you can bring into a program as if you wrote them yourself
- A module is a Python file containing functions, classes, or variables.
- Helps organize code and reuse it in multiple programs
- Types of modules:
  - Built-in modules: math, random, datetime
  - External modules: installed via pip, e.g., pandas

Code organized in modules

```
greetings.py

def greet():
  print("Hello,
    Python!")

main()
```

```
main.py

import greetings

greetings.greet()
```

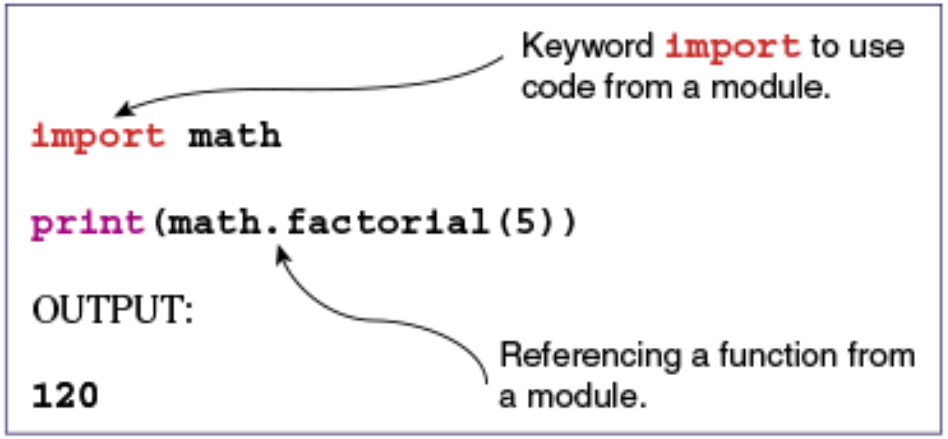
Importing Modules: Using import

- Creating an access to code from an external module is called importing.
- General syntax: `import <module_name>`
- Using import is like copying all the code from a module into your code

```python
# Importing various built-in modules using the 'import' keyword
import random
import math
import time

# Generate and print a random integer between 0 and 50 (inclusive)
print(random.randint(0, 50))

# Calculate and print the square root of 25
print(math.sqrt(25))

# Print the current date and time as a readable string
print(time.ctime())
```

Output

```
36
5.0
Thu Aug 21 04:57:35 2025
```



Ways to use built-in modules

```python
# Import the built-in random module
import random

# Import the built-in math module with an alias 'm'
import math as m

# Import only the 'ctime' function from the time module
from time import ctime

# Generate and print a random integer between 0 and 50 (inclusive)
print(random.randint(0, 50))

# Calculate and print the square root of 25 using the math module alias 'm'
print(m.sqrt(25))

# Print the current date and time as a readable string
print(ctime())
```

Output

```
38
5.0
Thu Aug 21 05:00:15 2025
```

Installing External Modules with `pip`

- External modules (e.g. pandas, numpy) can be installed using `pip`

```
pip install pandas
```

- Run this command in terminal / command prompt.
- After installation, you can `import pandas` in your python program

External module: Pandas

- Pandas is an external module for data manipulation

```python
import pandas as pd

# Create a simple DataFrame
data = {'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [25, 30, 22]}

df = pd.DataFrame(data)

print(df)
```

Output

```
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   22
```

Useful Modules in Python: Built-in Modules

- Some built-in modules such as `math` and `random`, must be imported first before use
- List of some functions in `math` module

Function	Description	Example
math.factorial()	Returns the multiplicative sum of a number (5 * 4 * 3 * 2 * 1).	math.factorial(5)
math.pow()	Returns the value of the first parameter raised to the power of the second parameter.	math.pow(4,3)
math.sqrt()	Returns the square root of a number.	math.sqrt(16)
math.cos()	Returns the cosine of the parameter, assuming the parameter is in radians.	math.cos(1.5)
math.pi	The mathematical constant π.	math.pi * radius * radius

Contd…

- List of some functions in the `random` module

Function	Description	Example
random.randint()	Return a random integer between the first parameter and the second parameter, including the second parameter.	random.randint(0,100) random.randint(0,1)
random.choice()	Return a random selection from an iterative sequence like a list.	random.choice(["red", "blue", "green"])
random.shuffle()	Shuffles the values in an iterative sequence; does not return anything but changes the value in the variable.	n = [x for x in range(10)] random.shuffle(n)
random.sample()	Returns a random unique selection of multiple values from an iterative sequence.	random.sample(["red", "blue", "green", "orange", "yellow"], 2)

Creating your own Modules

- To create a module, place your own .py file in the same folder as the program that will use the module
- The `main()` Function
  - Code in modules should be placed within functions so that the code only executes when those functions are called Module code outside of functions runs automatically when the module is imported

mymodule.py

```
def calculate_area(length, width):
    print("Area:", length*width)
```

main.py

```
import mymodule
mymodule.calculate_area(4, 6)  # Output: Area: 24
```

Next Week – What's Coming Up?

- Understand the difference between object-oriented programming (OOP) and procedural programming
- Exploring concepts of objects and classes in OOP
- Understanding the use of static elements in a Class
- Characteristics of Objects in OOP

CIHE

CIHE Australia